



B.TECH – S7 CSE – 23CSE401

Knowledge Representation and Reasoning

Unit 3 - Predicate Logic & Resolution • Structured Knowledge • Rule-Based Inference

Fuzzy Reasoning & Certainty Factors • Bayesian Networks • Dempster–Shafer Theory • Applied Case Studies

Introduction to Knowledge Representation

- Knowledge Representation (KR) is the field of AI concerned with encoding facts, rules, and relationships about the world into structures a machine can reason over.
- **A representation scheme must support four functions:**
 - Acquisition — acquiring and encoding new facts
 - Retrieval — efficiently locating relevant knowledge
 - Reasoning — deriving new facts from existing ones
 - Explanation — justifying conclusions to a user
- A good scheme balances expressive adequacy (can it say what we need?) against reasoning efficiency (can conclusions be drawn quickly?).

Four Pillars of KR

Logical
(Predicate Calculus)

Procedural
(Rules / Production)

Network
(Semantic Nets)

Structured
(Frames & Scripts)

Why Knowledge Representation Matters



Enables Automated Reasoning

Structured knowledge lets an inference engine derive new, previously unstated facts from what is already known.



Supports Explanation

Explicit representations let a system justify its conclusions — essential for trust in expert and decision-support systems.



Scales Beyond Hand-Coded Logic

Reusable structures (frames, rules, networks) let large knowledge bases grow without an explosion of ad-hoc code.



Separates Knowledge from Control

Declarative knowledge can be updated independently of the inference strategy that uses it — a core AI design principle.

Approaches to Knowledge Representation

Logical

Predicate calculus, propositional logic — precise, supports formal proof (resolution).

Procedural

Production rules: IF condition THEN action — knowledge embedded in executable procedures.

Network

Semantic networks — nodes (concepts) and labeled edges (relations) form a graph.

Structured

Frames and scripts — bundle related attributes and stereotypic event sequences together.

Introduction to Predicate Calculus

- Propositional logic can state facts but cannot express relationships between objects or generalize over classes — predicate calculus fixes this.
- A predicate is a function that maps objects to true/false, e.g. Likes(Father, Son).
- **Building blocks:**
 - Constants — specific objects: Father, Son, 5
 - Variables — stand for arbitrary objects: x , y
 - Functions — map objects to objects: FatherOf(x)
 - Predicates — relations that hold or not: Loves(x , y)
 - Quantifiers — $\forall$ (for all), $\exists$ (there exists)
 - Connectives — $\wedge$, $\vee$, $\neg$, $\rightarrow$, $\leftrightarrow$ combine formulas

Predicate Calculus — Syntax & Components

Element	Symbolic Form	Meaning
Term	$x, \text{John}, \text{FatherOf}(x)$	Refers to an object in the domain
Predicate	$\text{Likes}(x, y)$	A relation that is true or false for given terms
Atomic formula	$\text{Likes}(\text{Father}, \text{Son})$	A predicate applied to specific terms
Universal quantifier	$\forall x P(x)$	P holds for every object x in the domain
Existential quantifier	$\exists x P(x)$	P holds for at least one object x
Connectives	$\wedge \vee \neg \rightarrow \leftrightarrow$	AND, OR, NOT, IMPLIES, IFF

A well-formed formula (WFF) is any legal combination of these elements built by the formation rules of first-order logic.

Quantifiers — Worked Examples

English: All students study AI.

FOL: $\forall x \text{ (Student}(x) \rightarrow \text{Studies}(x, \text{AI}))}$

English: Some student has read every book.

FOL: $\exists x \text{ (Student}(x) \wedge \forall y \text{ (Book}(y) \rightarrow \text{Read}(x, y)))}$

English: No student fails every exam.

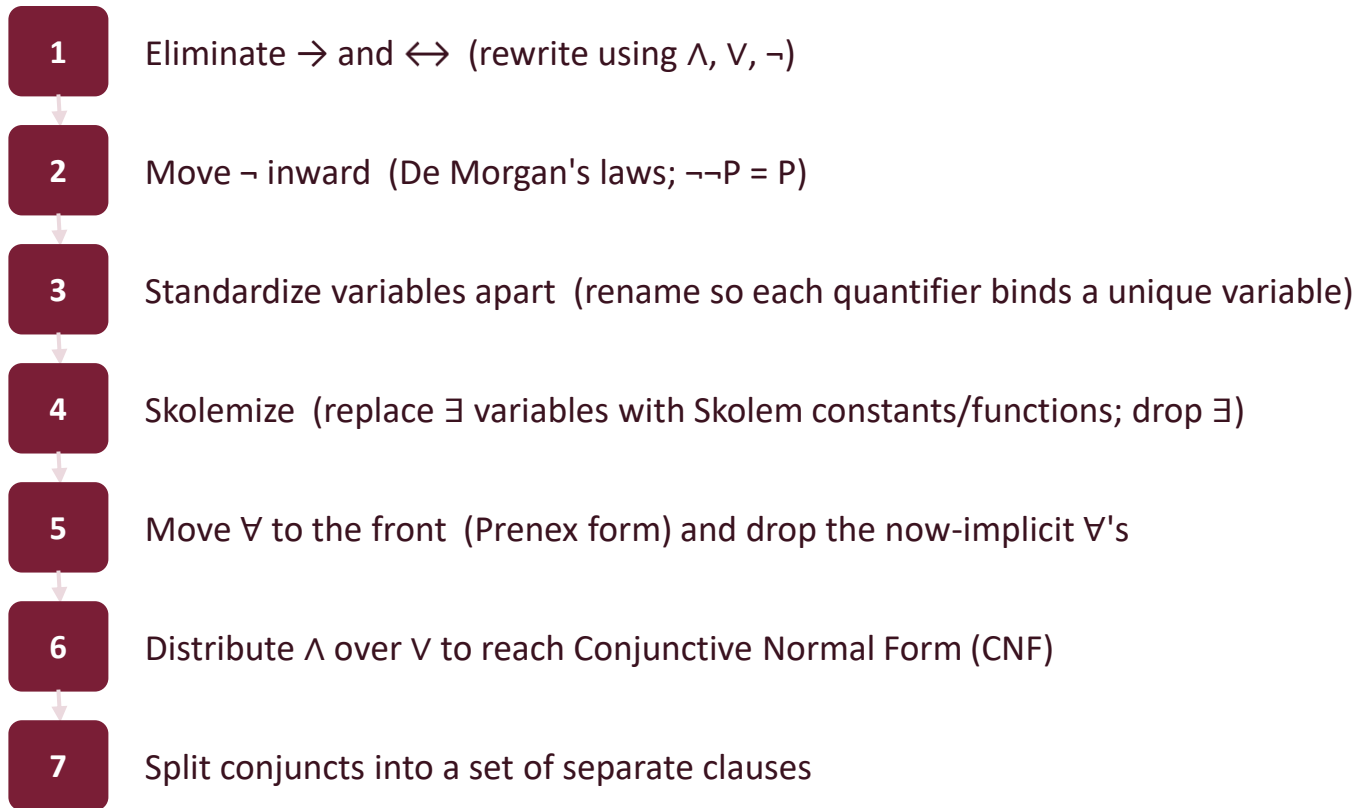
FOL: $\neg \exists x \text{ (Student}(x) \wedge \forall y \text{ (Exam}(y) \rightarrow \text{Fails}(x, y)))}$

English: Every parent loves their child.

FOL: $\forall x \forall y \text{ (Parent}(x, y) \rightarrow \text{Loves}(x, y))}$

From Well-Formed Formulas to Clausal Form

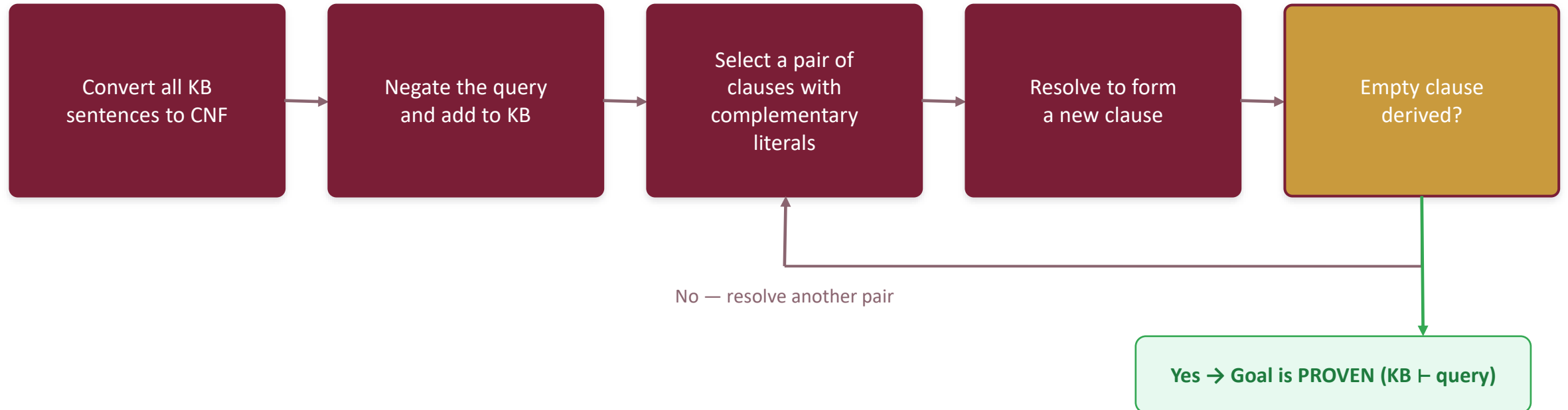
Resolution requires formulas in clausal form. The standard conversion pipeline:



Resolution — Concept

- Resolution is a single, sound inference rule that operates on clauses (disjunctions of literals) in CNF.
 - Resolution rule: from $(A \vee B)$ and $(\neg B \vee C)$, infer $(A \vee C)$ — the complementary literal B is cancelled.
 - **Proof by refutation:**
 - 1. Negate the goal and convert it to clause form.
 - 2. Add it to the knowledge base's clause set.
 - 3. Repeatedly resolve clause pairs.
 - 4. If the empty clause ($\square$) is derived, the original goal is proven — the negated goal made the KB inconsistent.
 - Completeness: resolution alone is refutation-complete for first-order logic — no separate axioms or rules needed.
-
- $(A \vee B) \rightarrow$ "A or B"
 - $(\neg B \vee C) \rightarrow$ "not B or C"
 - **Infer $(A \vee C) \rightarrow$ "therefore, A or C"**

Resolution Algorithm — Flow



If no complementary pair remains and the empty clause was never derived, the query does not follow from the KB.

Resolution — Worked Example

Knowledge base:

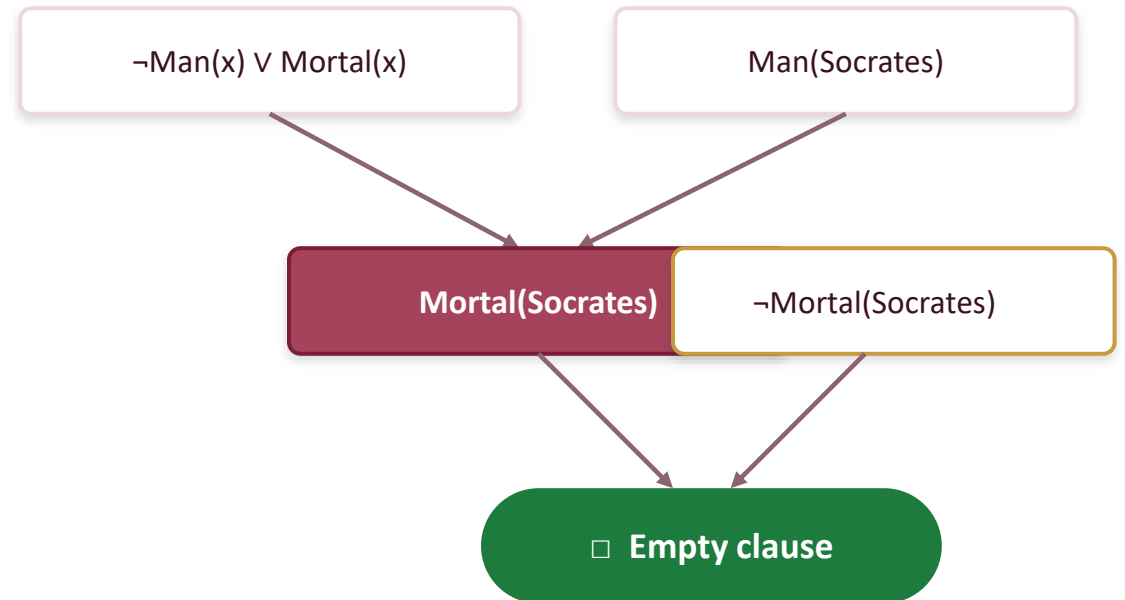
1. $\forall x \text{ (Man}(x) \rightarrow \text{Mortal}(x))$
2. $\text{Man}(\text{Socrates})$

Query: $\text{Mortal}(\text{Socrates})$?

$\neg \text{Man}(x) \vee \text{Mortal}(x)$

$\text{Man}(\text{Socrates})$

$\neg \text{Mortal}(\text{Socrates}) \leftarrow$ negated query



Contradiction found $\rightarrow$ $\text{Mortal}(\text{Socrates})$ is proven true.

Use of Predicate Calculus in AI Systems



Automated Theorem Proving

Resolution-based provers verify mathematical statements and check program correctness.



Logic Programming (Prolog)

Programs are written as sets of predicate-calculus facts and rules, executed via resolution/unification.



Expert System Knowledge Bases

Domain facts and inference rules are expressed as predicates for consistent, checkable reasoning.



Semantic Web & Ontologies

Description logics — a restricted predicate calculus — underpin OWL ontologies and RDF-based reasoning.

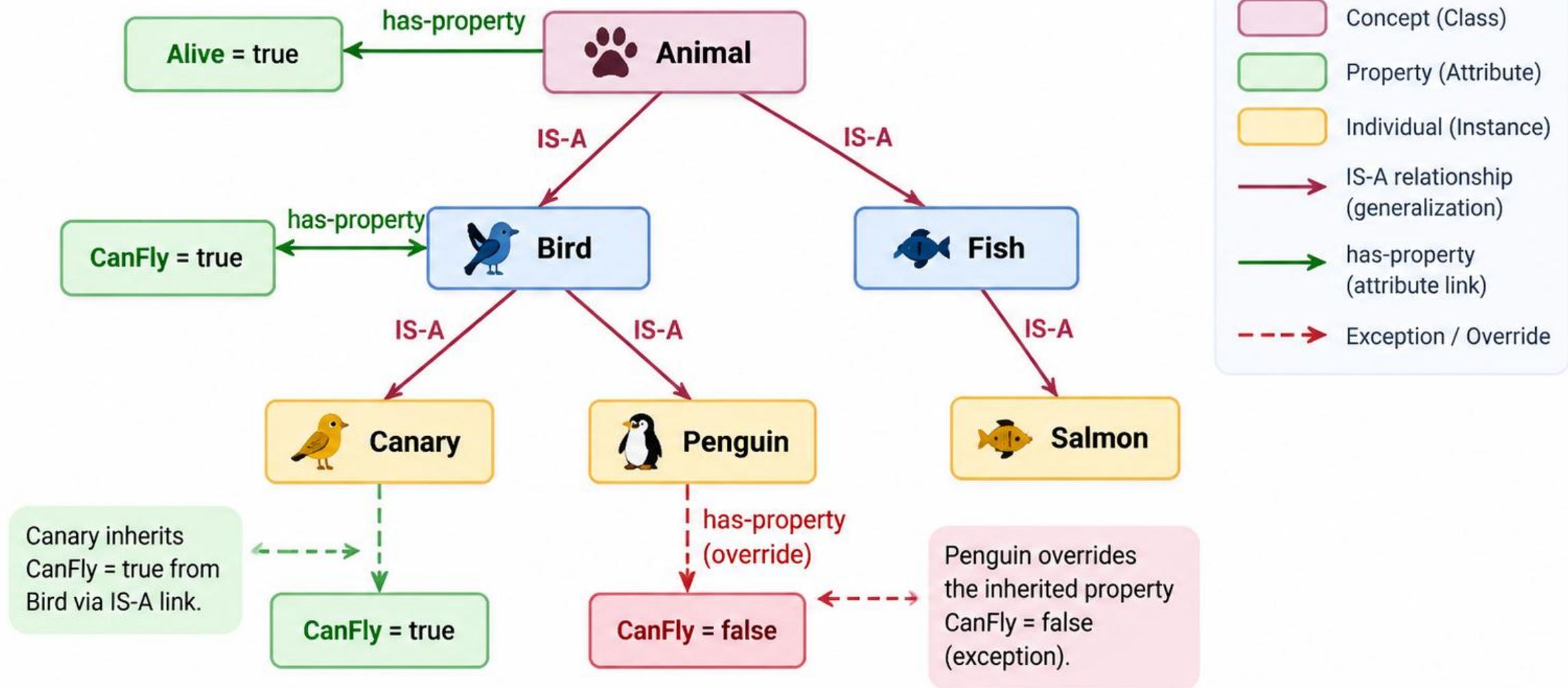
Knowledge Representation Using Other Logics

Logic	Adds / Changes	Typical Use
Propositional logic	No variables/quantifiers — whole statements only	Simple rule bases, circuit verification
Predicate (first-order) logic	Objects, relations, quantifiers	General-purpose KR, theorem proving
Modal logic	Operators for necessity/possibility ($\Box$, $\Diamond$)	Reasoning about belief, knowledge, time
Temporal logic	Operators over sequences of states/time	Verifying reactive & concurrent systems
Fuzzy logic	Degrees of truth in $[0,1]$ instead of $\{T,F\}$	Reasoning with vague, imprecise concepts
Non-monotonic logic	Conclusions can be withdrawn with new info	Default reasoning, exceptions to rules

Semantic Networks

- A semantic network is a directed graph: nodes represent objects/concepts, labeled arcs represent relationships between them.
- **Common relation types:**
 - IS-A — class/subclass membership (Canary IS-A Bird)
 - HAS-A / PART-OF — composition (Car HAS-A Engine)
 - Property links — attributes (Canary — color → Yellow)
- Inheritance: properties defined on a general node (Bird → CanFly) are inherited by all its specializations (Canary), avoiding redundant storage.
- Reasoning is done by graph traversal — following IS-A links upward to find inherited properties.

Semantic Network — Worked Example



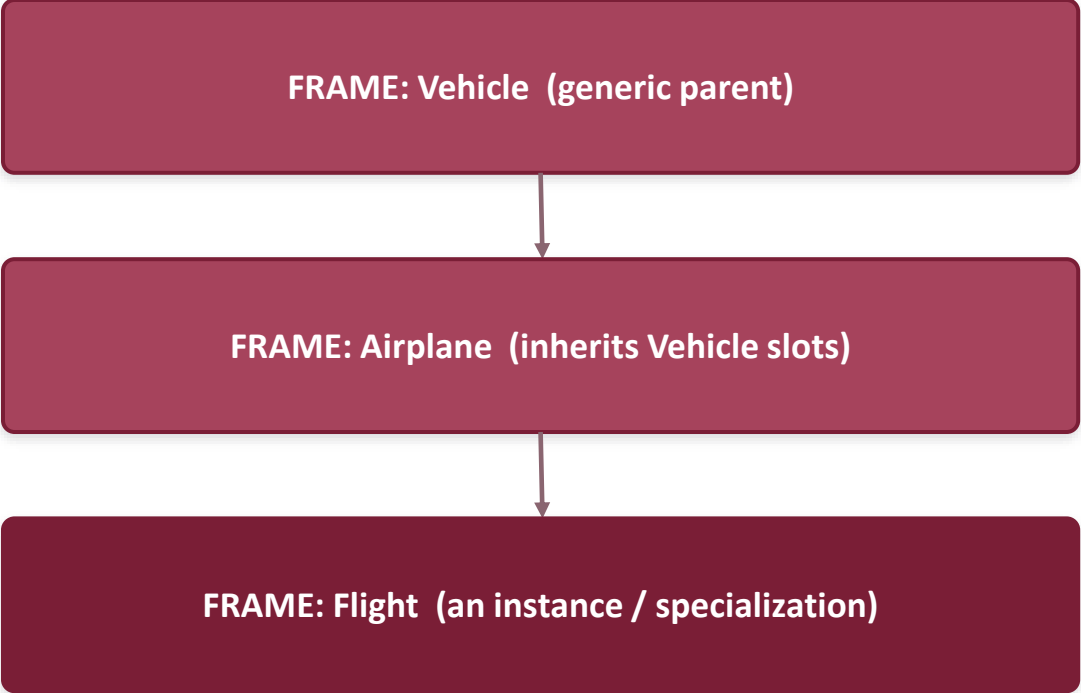
Inheritance allows concepts to obtain properties from parent nodes; exceptions override inherited properties.

Frames

- A frame is a data structure that bundles all the knowledge about a stereotyped object or situation into named slots.
- **Each slot holds:**
 - A value (fact), a default value, or a procedure (“demon”) that computes/updates it
- Frames are organized into hierarchies (like semantic nets) so specific frames inherit slots from general ones.
- Facets attached to a slot can restrict its type, trigger procedures IF-NEEDED (compute on access) or IF-ADDED (run when a value is stored).
- Frames excel at representing structured, expectation-driven knowledge — e.g. “a typical classroom has walls, desks, a teacher.”

Frame Example — Slots & Fillers

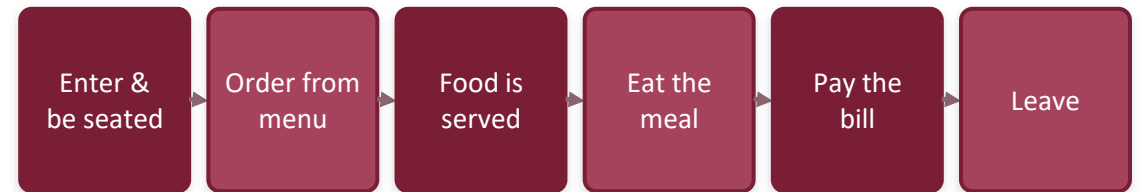
FRAME: Flight	
Airline	Value: "IndiGo"
Origin	Value: "Delhi"
Destination	Value: "Chennai"
Departure-Time	Default: 09:00
Duration	IF-NEEDED: compute(dist, speed)
Status	IF-ADDED: notify(passenger)



Slots not explicitly filled in Flight (e.g. Fuel-Capacity) are inherited from Airplane or Vehicle.

Scripts

- A script represents a stereotyped sequence of events in a well-understood situation — e.g. dining at a restaurant.
- **Standard components:**
 - Entry conditions — must hold before the script applies
 - Props — objects involved (tables, menu, food, bill)
 - Roles — participants (customer, waiter, cook, cashier)
 - Scenes — ordered sub-sequences of events
 - Results — conditions true after the script completes
- Scripts let a system fill in unstated but expected details — e.g. inferring the customer paid, even if the text never says so.



Scenes of the RESTAURANT script

Given only “John went to a restaurant and left a big tip,” a script-based reasoner infers he ordered, ate, and paid — none of which was stated.

Semantic Nets vs. Frames vs. Scripts

Aspect	Semantic Network	Frame	Script
Structure	Graph of nodes & labeled arcs	Object with named slots	Ordered sequence of scenes
Best for	Taxonomies, IS-A hierarchies	Stereotyped objects & attributes	Stereotyped event sequences
Inheritance	Via IS-A arc traversal	Via frame hierarchy	Limited — via sub-scripts
Procedural attachment	Rare	IF-NEEDED / IF-ADDED demons	Implicit in scene ordering
Example use	“A canary is a bird”	“A flight has an airline, origin...”	“Dining at a restaurant”

From Representation to Inference

- Representing knowledge is only half the problem — an inference engine must use it to derive new facts, answer queries, and support decisions.
- This part covers two representation-linked inference architectures (production systems, frame-based systems) and the search strategies that drive them (forward chaining, backward chaining, rule-value ordering).
- Central design question: given a large rule/fact base, in what order should rules fire to reach a goal efficiently and correctly?

**Production
Systems**

**Frame-Based
Inference**

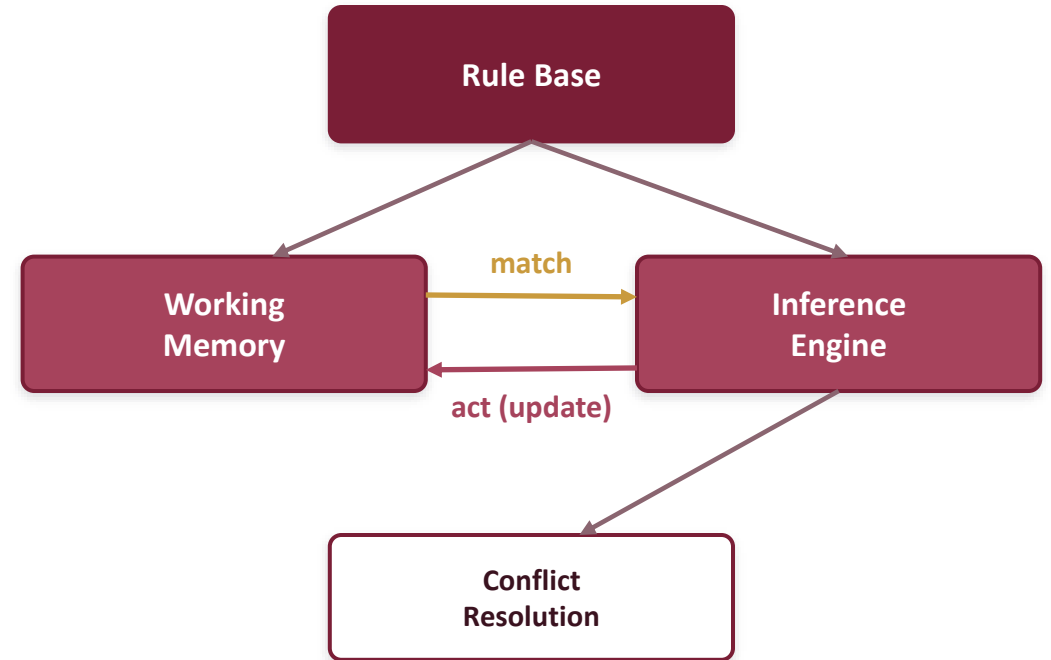
**Forward
Chaining**

**Backward
Chaining**

**Rule-Value
Approach**

Production-Based Systems — Architecture

- A production system separates knowledge into IF–THEN rules and applies them via a general control cycle.
- **Three components:**
 - Rule base (production memory) — the set of IF–THEN rules
 - Working memory — current facts describing the problem state
 - Inference (recognize–act) engine — matches rules against working memory and fires them
- The recognize–act cycle repeats: match → select (conflict resolution) → act — until the goal is reached or no rule matches.



The cycle repeats until working memory satisfies the goal or no rule's IF-part is matched.

Production System — Worked Example

Working memory: Temperature = High, Cough = Yes

R1: IF Temperature = High AND Cough = Yes THEN Suspect = Flu



R2: IF Suspect = Flu AND Age > 60 THEN Risk = High



R3: IF Risk = High THEN Recommend = "See a doctor immediately"

Fires R1 → R2 → R3 in sequence, chaining forward from data to a recommendation.

Frame-Based Systems — Inference

- Inference in a frame system happens through slot access, inheritance, and procedural attachments rather than explicit rule chaining.
- **Key mechanisms:**
 - Inheritance — an unfilled slot is resolved by walking up the frame hierarchy to a parent frame's value or default.
 - IF-NEEDED procedures — triggered when a slot is read but has no value; computes it on demand.
 - IF-ADDED / IF-REMOVED procedures — triggered when a slot value changes, propagating consequences to other frames.
 - Matching — an input situation is classified by finding the best-fitting frame (“frame selection”), after which its expectations guide further processing.
- Because frames combine declarative slots with attached procedures, they blend declarative and procedural knowledge representation.

Inference Mechanisms — Overview

Mechanism	Direction	Best Suited For
Forward chaining	Data → Goal	Monitoring, data-driven situations (many facts, few goals)
Backward chaining	Goal → Data	Diagnosis, goal-driven queries (specific hypothesis to test)
Rule-value approach	Priority-ordered	Large rule sets where some rules are far more useful/certain
Frame-based (inheritance)	Hierarchy traversal	Structured, stereotyped object knowledge

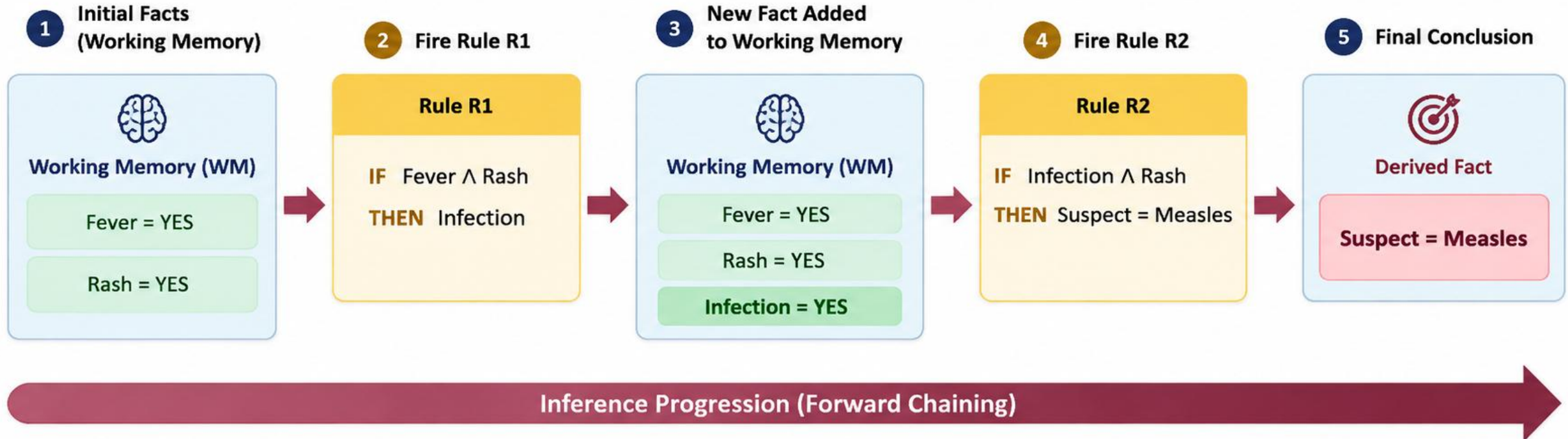
Real systems often combine strategies — e.g. forward chaining to gather evidence, then backward chaining to confirm a specific hypothesis.

Forward Chaining

- Data-driven reasoning: start from known facts in working memory and fire any rule whose IF-part is satisfied, adding new facts.
- Repeat until the goal fact appears, or no further rule can fire.
- **Characteristics:**
 - Well suited when there are many possible conclusions but a fixed, small set of starting facts.
 - Can generate many irrelevant intermediate facts if not carefully controlled.
 - Typical of monitoring and data-interpretation systems (e.g. sensor networks, process control).
- Also called “bottom-up” reasoning: from evidence up to conclusions.

Forward Chaining — Example Trace

Forward chaining starts from known facts and applies rules to infer new facts until a goal is reached.

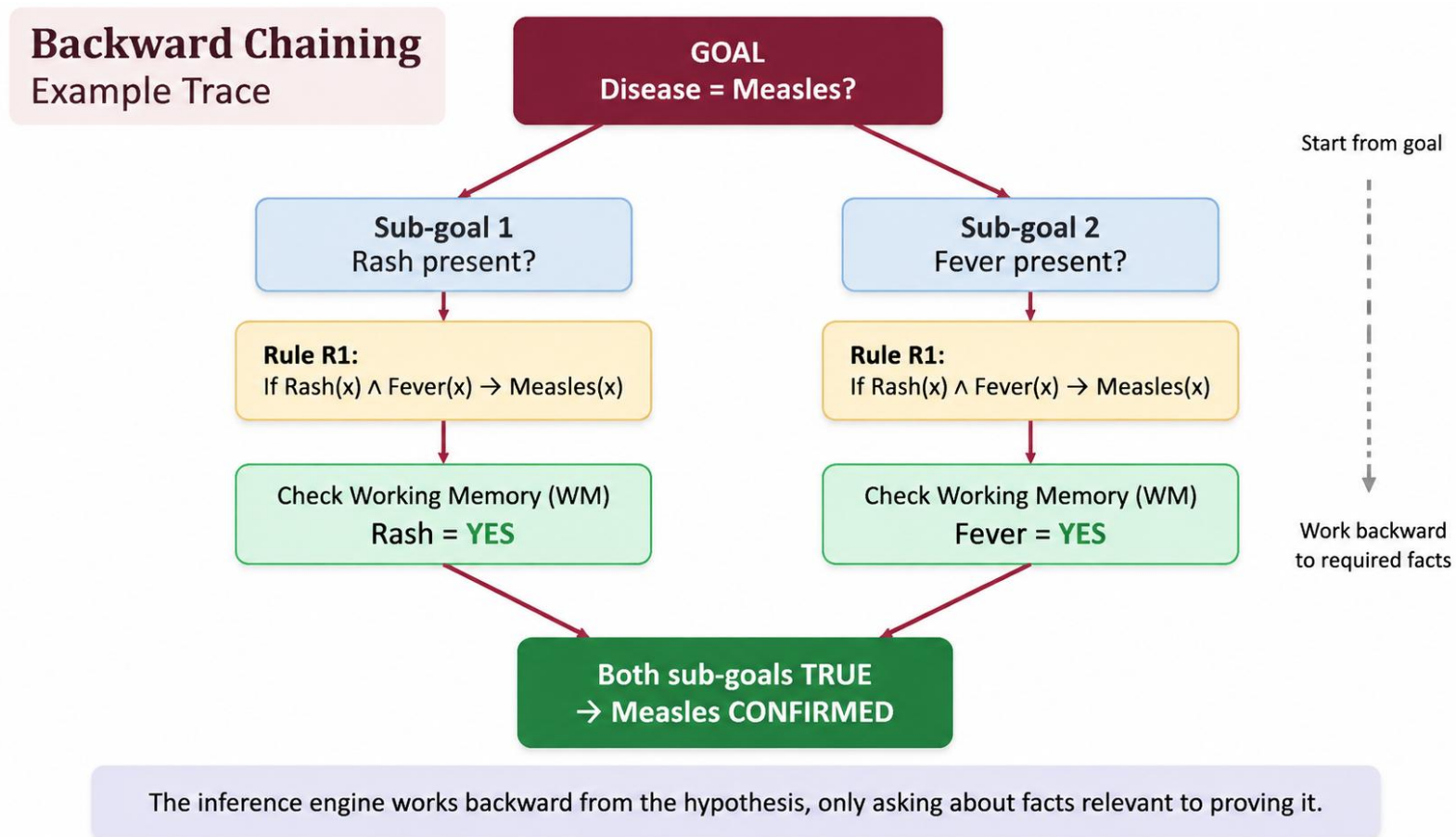


Starting from the observed facts, the engine repeatedly fires matching rules, accumulating new facts until the goal "Suspect = Measles" is reached.

Backward Chaining

- Goal-driven reasoning: start from a hypothesis (the goal) and search for rules that would conclude it, then recursively try to prove their premises.
- If a premise is not already a known fact, it becomes a new sub-goal to prove.
- **Characteristics:**
 - Efficient when there is one specific hypothesis to confirm or deny out of many possibilities.
 - Avoids deriving irrelevant facts — only explores what's relevant to the goal.
 - Typical of diagnostic expert systems (e.g. MYCIN): “Is the patient's infection bacterial?” drives the search.
- Also called “top-down” reasoning: from a hypothesis down to supporting evidence.

Backward Chaining — Example Trace



Forward Chaining vs. Backward Chaining

Aspect	Forward Chaining	Backward Chaining
Direction	Facts → Conclusions (data-driven)	Goal → Facts (goal-driven)
Starts from	All known facts in working memory	A specific hypothesis to test
Best when	Many outcomes, few starting facts	Few hypotheses, many possible facts
Typical system	Monitoring, planning, configuration	Diagnosis (e.g. MYCIN), troubleshooting
Risk	Generates irrelevant intermediate facts	May re-derive the same sub-goal repeatedly
Also known as	Bottom-up / data-driven	Top-down / goal-driven

The Rule-Value Approach

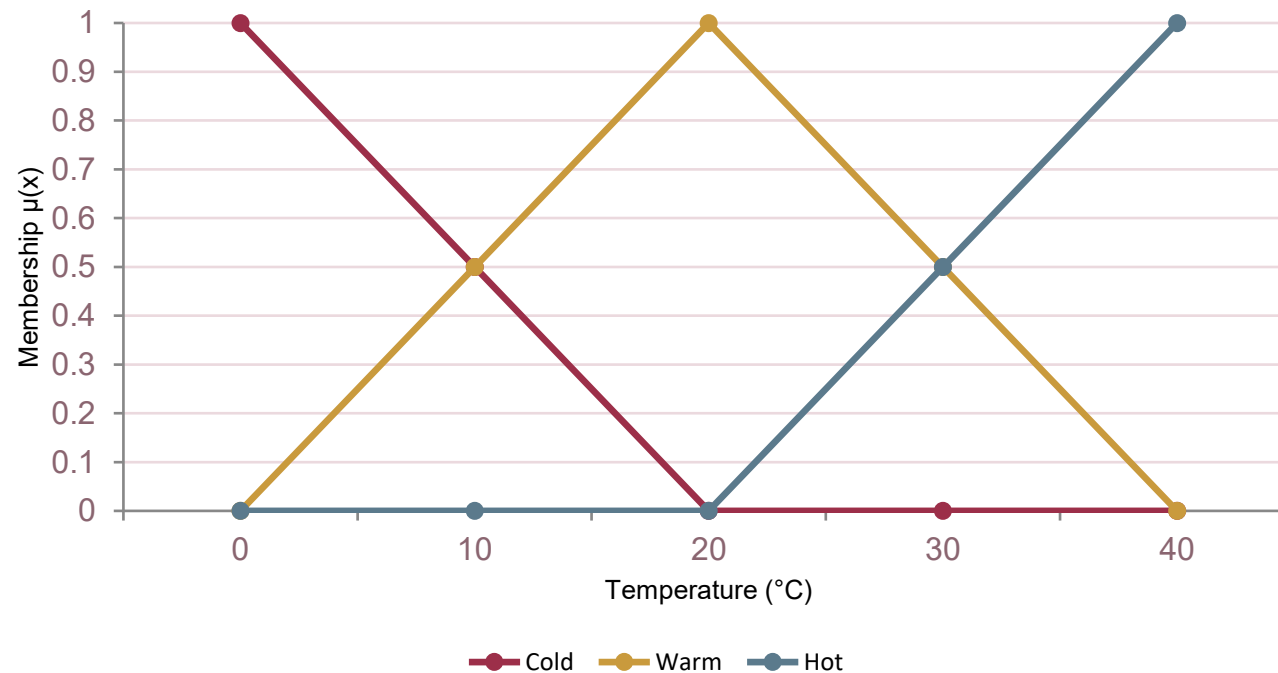
- When many rules could fire, conflict resolution decides which one to apply first — the rule-value approach assigns each rule a numeric “value” reflecting its usefulness.
- **A rule's value can be based on:**
 - Certainty / reliability of its conclusion in the domain
 - Cost of gathering the evidence it needs
 - Specificity — more specific rules often outrank general ones
 - Historical success rate (learned or tuned by experts)
- The engine sorts matching rules by value and fires the highest-value rule first, improving both speed and the quality of early conclusions.
- This differs from pure forward/backward chaining, which typically use simple recency or order-of-appearance for conflict resolution.

Fuzzy Reasoning — Introduction

- Classical (“crisp”) sets require exact membership: an object is either fully IN or fully OUT of a set.
- Fuzzy sets (Zadeh, 1965) allow partial membership: a membership function $\mu(x) \in [0, 1]$ expresses the degree to which x belongs.
- Example: “Temperature is Hot” isn't strictly true/false — 30°C might be Hot with $\mu = 0.7$, Warm with $\mu = 0.3$.
- Fuzzy reasoning applies logical operators (AND = min, OR = max, NOT = $1 - \mu$) directly to these degrees, letting rules like “IF temperature is Hot THEN fan speed is High” operate smoothly rather than as a hard switch.

Fuzzy Sets & Membership Functions

Membership functions for “Temperature” (°C)



Reading the chart

- At 22°C: $\mu(\text{Warm}) \approx 0.8$, $\mu(\text{Hot}) \approx 0.1$ — the temperature is mostly Warm.
- Overlapping regions let a value belong to two sets simultaneously (e.g. 25°C is both Warm and Hot).
- Fuzzy rules fire in parallel, weighted by these degrees, then outputs are combined (defuzzified) into one crisp action.

Certainty Factors (CF)

- Developed for MYCIN, certainty factors quantify belief in a hypothesis without full probabilistic machinery.
- **$CF(H, E) \in [-1, +1]$ measures confidence that evidence E supports hypothesis H:**
 - +1 — E confirms H with certainty
 - -1 — E disconfirms H with certainty
 - 0 — no evidence either way
- CF is composed from measures of belief and disbelief: $CF = MB(H,E) - MD(H,E)$
- Rules attach a CF to their conclusion, e.g. “IF fever THEN infection (CF = 0.6)” — reflecting the rule's own reliability.

Certainty Factors — Combining Evidence

Situation	Combination Formula
Two rules support the SAME hypothesis (both CFs positive)	$CF(H) = CF1 + CF2 \times (1 - CF1)$
Two rules conflict (one positive, one negative)	$CF(H) = (CF1 + CF2) / (1 - \min(CF1 , CF2))$
Both rules disconfirm (both CFs negative)	$CF(H) = CF1 + CF2 \times (1 + CF1)$
Rule premise itself is uncertain (conjunctive AND)	$CF(\text{premise}) = \min(CF1, CF2, \dots)$
Rule premise itself is uncertain (disjunctive OR)	$CF(\text{premise}) = \max(CF1, CF2, \dots)$

Example: two independent rules each give $CF = 0.6$ for “Infection.” Combined: $0.6 + 0.6 \times (1 - 0.6) = 0.84$ — confidence grows but never exceeds 1.

Bayesian Theory — Bayes' Theorem

- Bayesian reasoning treats uncertainty as formal probability, updated as new evidence arrives.
- **Bayes' Theorem:**
- Lets us compute the posterior probability of a hypothesis H given evidence E , from the likelihood and the prior.
 - $P(H)$ — prior belief in H before seeing evidence
 - $P(E|H)$ — likelihood: how probable the evidence is if H is true
 - $P(E)$ — total probability of the evidence across all hypotheses
 - $P(H|E)$ — posterior: updated belief in H after observing E
- Unlike certainty factors, Bayesian updating has a rigorous mathematical foundation and composes correctly across many pieces of evidence — at the cost of needing full probability tables.

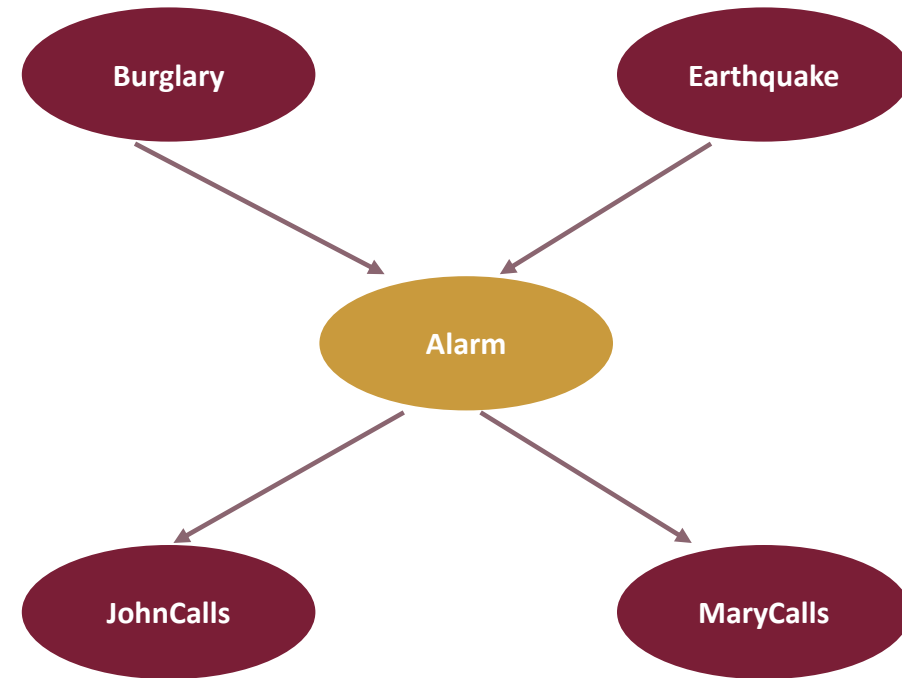
$$P(H|E) = \frac{P(E|H) \cdot P(H)}{P(E)}$$

Worked example — medical test

- $P(\text{Disease}) = 0.01$ (prior)
- $P(\text{Positive} | \text{Disease}) = 0.9$ (sensitivity)
- $P(\text{Positive}) = 0.098$ (total probability)
- $P(\text{Disease} | \text{Positive}) \approx 0.092$

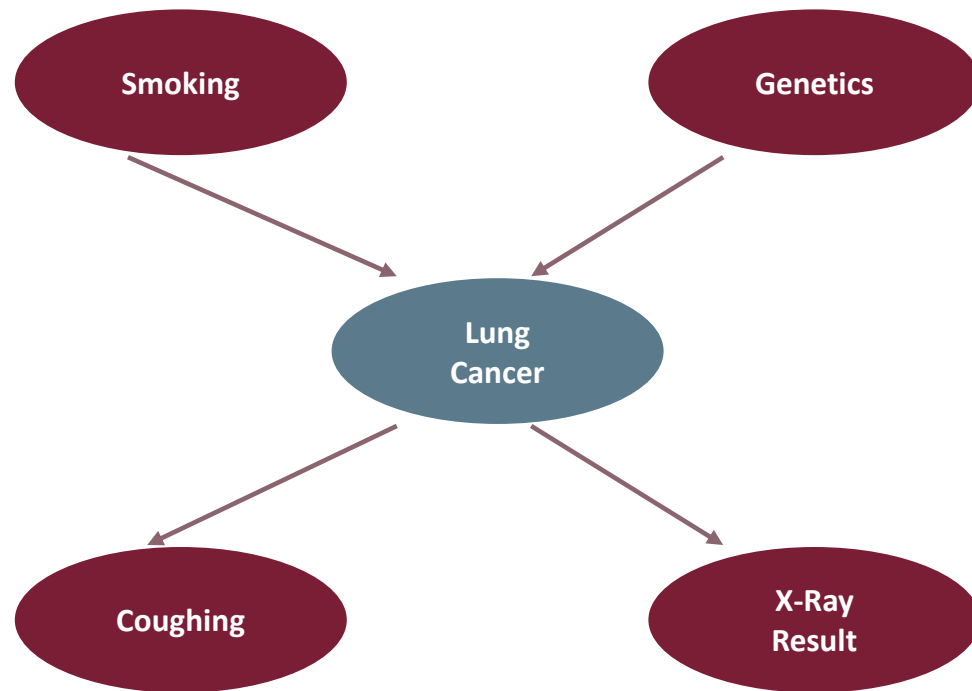
Bayesian Networks — Structure

- A Bayesian network is a directed acyclic graph (DAG) where nodes are random variables and edges encode direct probabilistic dependence.
- Each node stores a Conditional Probability Table (CPT) giving $P(\text{node} \mid \text{its parents})$.
- Key property: a node is conditionally independent of its non-descendants given its parents — this lets the full joint distribution factor into small, local CPTs.
- Inference (e.g. “what's $P(\text{Alarm} \mid \text{Burglary observed})$?”) propagates evidence through the graph using the chain rule of probability.



$P(\text{JohnCalls} \mid \text{Alarm})$ and $P(\text{MaryCalls} \mid \text{Alarm})$ are conditionally independent given Alarm.

Bayesian Network — Medical Diagnosis Example



CPT: $P(\text{Lung Cancer} \mid \text{Smoking, Genetics})$

Smoking	Genetics	P(Cancer)
Yes	Yes	0.75
Yes	No	0.45
No	Yes	0.20
No	No	0.02

- Evidence: Coughing = Yes, X-Ray = Positive
- Query: $P(\text{Lung Cancer} \mid \text{Coughing, X-Ray})$?
- The network propagates both pieces of evidence upward through Bayes' rule to update belief in Lung Cancer — combining weak individual signals into a stronger diagnosis.

Dempster–Shafer Theory of Evidence

- Generalizes Bayesian probability to explicitly represent ignorance — the case where evidence is simply insufficient to decide.
- **Core concepts:**
 - Frame of discernment (Θ) — the set of all mutually exclusive hypotheses
 - Mass function $m(A)$ — belief committed exactly to subset A of Θ (not further divisible)
 - Belief function $\text{Bel}(A)$ — total support for A (sum of mass of A and its subsets)
 - Plausibility $\text{Pl}(A)$ — how much A could be believed if further evidence arrived (upper bound)
- Dempster's rule of combination merges independent mass functions from multiple sources into one, focusing belief as evidence accumulates.
- Advantage over Bayesian methods: doesn't force a prior probability onto hypotheses with no evidence — explicitly models “unknown.”

Dempster-Shafer — Worked Example

Frame of discernment $\Theta = \{\text{Bacterial}, \text{Viral}, \text{Fungal}\}$ for an infection's cause.

Source	Hypothesis Set	Mass $m(A)$
Lab Test 1	{Bacterial}	0.6
Lab Test 1	Θ (unknown)	0.4
Lab Test 2	{Bacterial, Viral}	0.5
Lab Test 2	Θ (unknown)	0.5

Combined via Dempster's Rule

- $m(\{\text{Bacterial}\})$ rises to ≈ 0.80
- $m(\{\text{Bacterial}, \text{Viral}\}) \approx 0.20$ (remaining ignorance narrows to this set)
- Belief in “Bacterial” grows because both independent tests point toward it — while unresolved mass still reflects real uncertainty.

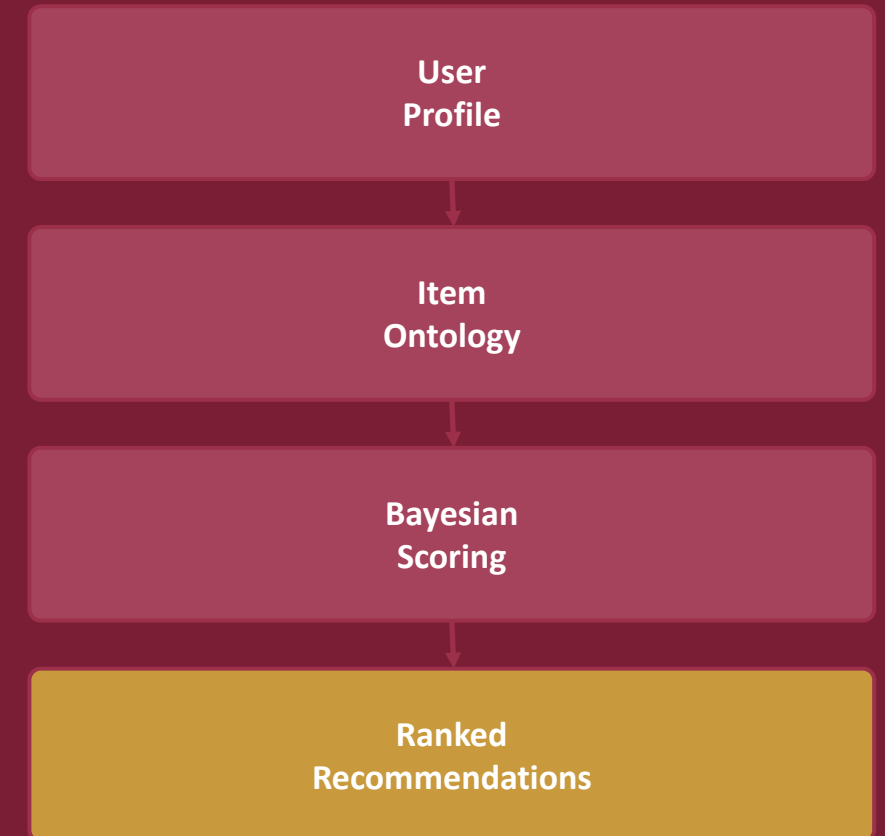
Contrast with Bayesian: DS keeps 0.20 explicitly “unresolved” rather than forcing it to split by an assumed prior between Bacterial and Viral.

Certainty Factors vs. Bayesian vs. Dempster–Shafer

Aspect	Certainty Factors	Bayesian Theory	Dempster–Shafer
Foundation	Heuristic, ad-hoc	Formal probability theory	Generalized probability (belief functions)
Represents ignorance?	Poorly	No — forces a prior	Yes — explicitly
Data required	Expert-assigned CFs	Full prior & conditional tables	Mass assignments per source
Computational cost	Low	Can be high (large CPTs)	Higher — combination is combinatorial
Best for	Quick expert-system rules	Well-understood probabilistic domains	Fusing multiple uncertain evidence sources

Recommendation Systems

- Recommendation engines apply KR & inference techniques to predict what a user will want next.
- **Knowledge sources combined:**
 - User–item interaction facts (ratings, clicks, purchases) — working-memory-like facts
 - Item ontologies / taxonomies — semantic-network style category structure
 - Bayesian models estimate $P(\text{user likes item} \mid \text{past behavior})$; rule-based filters apply hard constraints (e.g. “in stock,” “age-appropriate”)
- Case study: Netflix/Amazon-style engines blend collaborative filtering (Bayesian-flavored) with content-based rules drawn from item attribute frames.



Knowledge-Based Systems in Computer Science



Cybersecurity — Intrusion Detection

Rule-based production systems and Bayesian classifiers flag anomalous network behavior using forward chaining over live traffic facts.



Software Engineering — Code Review Assistants

Frame-based representations of code patterns match against ASTs; rule bases encode style and security best practices.



Compilers & Program Analysis

Predicate-logic-based static analyzers (e.g. Datalog engines) use resolution-like fixpoint inference to detect bugs.



IT Operations — Automated Troubleshooting

Backward chaining from a fault symptom to root cause, using a frame hierarchy of system components.